

Informe de implementación de tests unitarios y de comportamiento

Sistema de Solicitudes Académicas - Formativo II PSP 2026

Preparado para revisión del proyecto

23 de junio de 2026

Propósito del informe

Este informe documenta la implementación de la suite de pruebas solicitada por el README del proyecto. Se asume la arquitectura indicada por el estudiante: existe un punto de composición tipo **AppConfig** que instancia las implementaciones concretas de infraestructura, crea el controlador y luego entrega dicho controlador a la vista. Bajo ese supuesto, la implementación de tests se concentra en reglas de dominio, casos de uso de aplicación, interacción con dependencias externas mediante Mockito, reglas arquitecturales con ArchUnit y publicación/recepción de eventos con Vert.x.

Índice

1 Criterio general de diseño de pruebas

La suite se diseñó con una regla central: **cada comportamiento se prueba en la capa donde vive**. Esto evita pruebas redundantes y mantiene la intención de cada test clara.

- Las validaciones de nombre, correo, tipo, descripción y cambio de estado pertenecen al **dominio**; por eso se prueban en `EstudianteTest` y `SolicitudDomainTest`.
- Los servicios de aplicación no vuelven a probar exhaustivamente cada validación interna del dominio. En ellos se verifica la **coordinación del caso de uso**: buscar, cambiar estado, guardar, publicar evento y notificar.
- Mockito se usa para aislar dependencias externas: repositorios, publicador de eventos y notificador externo.
- Vert.x se prueba verificando el comportamiento observable del `EventBus`: publicación del payload y recepción por un consumidor.
- ArchUnit se usa para comprobar que la arquitectura respetada por el código coincide con la separación `view/controller/application/domain/infra`.

2 Supuestos arquitecturales usados

El informe asume la siguiente composición:

Listing 1: Flujo arquitectural asumido

```
AppConfig
-> instancia EstudianteRepository como EstudiantePort
-> instancia SolicitudRepository como SolicitudPort
-> instancia ServicioCorreoExterno como NotificadorExterno
-> instancia VertxPublicadorEventosSolicitud como PublicadorEventosSolicitud
-> crea SolicitudController
Main
-> crea la vista usando el controller ya construido
View
-> solo delega acciones al Controller
Controller
-> coordina servicios de aplicacin
Application Service
-> depende de puertos e interfaces
Infra
-> implementa puertos e interfaces
```

Bajo esta decisión, los tests arquitecturales no se diseñan para permitir que la vista o el controlador importen `infra`, sino para detectar ese error. Si el código real mantiene imports de infraestructura en `view` o `controller`, esos tests deben fallar, lo cual sería correcto.

3 Matriz de cobertura respecto del README

N.	Caso sugerido por README	Test implementado
----	--------------------------	-------------------

1	Creación válida de estudiante	<code>EstudianteTest.creacionValidaDeEstudianteDebeConse</code>
2	Rechazo de estudiante con nombre vacío	<code>EstudianteTest.noDebeAceptarNombreVacio</code> y casos nulo/espacios
3	Rechazo de estudiante con correo vacío	<code>EstudianteTest.noDebeAceptarCorreoVacio</code> y casos nulo/espacios
4	Creación válida de solicitud	<code>SolicitudDomainTest.creacionValidaDeSolicitudDebeC</code>
5	Rechazo de solicitud con estudiante nulo	<code>SolicitudDomainTest.noDebeAceptarEstudianteNulo</code>
6	Rechazo de solicitud con tipo vacío	<code>SolicitudDomainTest.noDebeAceptarTipoVacio</code> y espacios/nulo
7	Rechazo de solicitud con descripción vacía	<code>SolicitudDomainTest.noDebeAceptarDescripcionVacía</code> y espacios/nulo
8	Estado inicial de solicitud nueva	<code>SolicitudDomainTest.solicitudNuevaDebeQuedarPendie</code>
9	Cambio válido de estado	<code>SolicitudDomainTest.cambioValidoDeEstadoDebeActual</code>
10	Rechazo de cambio cuando la solicitud ya está finalizada	<code>SolicitudDomainTest.solicitudAprobadaNoDebeCambiar</code> y <code>solicitudRechazadaNoDebeCambiarEstado</code>
11	Rechazo de cambio con estado nulo	<code>SolicitudDomainTest.noDebeAceptarEstadoNuloAlCambi</code>
12	Caso de uso de cambio de estado desde aplicación	<code>SolicitudServiceMockitoTest.cambiarEstadoDebeGuard</code>
13	Caso de solicitud inexistente	<code>SolicitudServiceMockitoTest.cambiarEstadoConSolic</code>
14	Publicación de evento al cambiar estado	<code>SolicitudServiceMockitoTest</code> con <code>ArgumentCaptor</code> ; además <code>VertxPublicadorEventosSolicitudTest</code>
15	Notificación externa al cambiar estado	<code>SolicitudServiceMockitoTest</code> verifica <code>notificador.enviarCorreo(...)</code>

4 Tests de dominio

4.1 EstudianteTest

Este test cubre las reglas propias de la entidad `Estudiante`. No se delega nada a repositorios ni servicios, porque la entidad debe ser verificable de forma aislada.

Listing 2: Estructura principal de `EstudianteTest`

```
@Test
public void creacionValidaDeEstudianteDebeConservarNombreYCorreo() {
    Estudiante estudiante = new Estudiante("Ana Torres", "ana@universidad.cl");

    assertEquals("Ana Torres", estudiante.getNombre());
    assertEquals("ana@universidad.cl", estudiante.getCorreo());
}

@Test(expected = IllegalArgumentException.class)
public void noDebeAceptarNombreVacio() {
    new Estudiante("", "ana@universidad.cl");
}

@Test(expected = IllegalArgumentException.class)
public void noDebeAceptarCorreoVacio() {
    new Estudiante("Ana Torres", "");
}
```

Además se implementaron variantes para null y cadenas compuestas únicamente por espacios. Esto es relevante porque la implementación del dominio usa `trim().isEmpty()`, por lo que no basta probar solamente la cadena vacía literal.

4.2 SolicitudDomainTest

Este test concentra las reglas de negocio de la entidad `Solicitud`: creación válida, campos obligatorios, estado inicial, cambio válido y rechazo de transiciones inválidas.

Listing 3: Cambio válido de estado en dominio

```
@Test
public void cambioValidoDeEstadoDebeActualizarEstadoYFecha() {
    Estudiante estudiante = new Estudiante("Ana Torres", "ana@universidad.cl");
    Solicitud solicitud = new Solicitud(estudiante, "Certificado", "Solicitud valida");

    solicitud.cambiarEstado(EstadoSolicitud.EN_REVISION);

    assertEquals(EstadoSolicitud.EN_REVISION, solicitud.getEstado());
    assertNotNull(solicitud.getFechaActualizacion());
}
```

El rechazo de cambios desde estados finales se prueba en dos casos separados: `APROBADA` y `RECHAZADA`. Esto aumenta precisión porque ambos valores dependen de `EstadoSolicitud.esFinal()`.

Listing 4: Rechazo de cambios desde estado final

```
@Test(expected = IllegalStateException.class)
public void solicitudAprobadaNoDebeCambiarEstado() {
    Estudiante estudiante = new Estudiante("Ana Torres", "ana@universidad.cl");
    Solicitud solicitud = new Solicitud(estudiante, "Certificado", "Solicitud valida");

    solicitud.cambiarEstado(EstadoSolicitud.APROBADA);
    solicitud.cambiarEstado(EstadoSolicitud.EN_REVISION);
}
```

4.3 EstadoSolicitudTest

Se agregó un test pequeño para cubrir el método `esFinal()`. Aunque es simple, es una regla de dominio usada por `Solicitud.cambiarEstado(...)`.

Listing 5: Estados finales y no finales

```
@Test
public void aprobadaYRechazadaDebenSerEstadosFinales() {
    assertTrue(EstadoSolicitud.APROBADA.esFinal());
    assertTrue(EstadoSolicitud.RECHAZADA.esFinal());
}

@Test
public void pendienteYEnRevisionNoDebenSerEstadosFinales() {
    assertFalse(EstadoSolicitud.PENDIENTE.esFinal());
    assertFalse(EstadoSolicitud.EN_REVISION.esFinal());
}
```

5 Tests de servicios de aplicación

5.1 EstudianteServiceTest

El objetivo del test no es volver a probar todos los errores de `Estudiante`; eso ya está cubierto en dominio. El servicio debe probarse como coordinador: crea la entidad y delega su persistencia al puerto `EstudiantePort`.

Listing 6: Verificación de delegación en `EstudiantePort`

```
@Test
public void guardarDebeCrearEstudianteYDelegarEnElPuerto() {
    EstudiantePort estudiantePort = mock(EstudiantePort.class);
    when(estudiantePort.guardar(any(Estudiante.class)))
        .thenAnswer(invocation -> invocation.getArgument(0));

    EstudianteService service = new EstudianteService(estudiantePort);

    Estudiante resultado = service.guardar("Ana Torres", "ana@universidad.cl");

    assertEquals("Ana Torres", resultado.getNombre());
    assertEquals("ana@universidad.cl", resultado.getCorreo());

    ArgumentCaptor<Estudiante> captor = ArgumentCaptor.forClass(Estudiante.class);
    verify(estudiantePort).guardar(captor.capture());
    assertEquals("Ana Torres", captor.getValue().getNombre());
    assertEquals("ana@universidad.cl", captor.getValue().getCorreo());
}
```

También se implementó un caso donde los datos son inválidos. Este test no pretende repetir toda la validación del dominio, sino verificar una consecuencia de aplicación: si la construcción de la entidad falla, el puerto no debe ser invocado.

Listing 7: Si el dominio rechaza datos, el puerto no debe usarse

```
@Test
public void guardarConDatosInvalidosNoDebeInvocarPuerto() {
    EstudiantePort estudiantePort = mock(EstudiantePort.class);
    EstudianteService service = new EstudianteService(estudiantePort);

    assertThrows(IllegalArgumentException.class,
        () -> service.guardar("", "ana@universidad.cl"));

    verifyNoInteractions(estudiantePort);
}
```

5.2 SolicitudServiceMockitoTest

Este es el test más importante del README, porque valida el caso de uso con dependencias externas reemplazadas por mocks. Se verifica lo siguiente:

1. Se busca la solicitud mediante `SolicitudPort`.
2. Se cambia el estado usando la regla de dominio.
3. Se guarda la solicitud modificada.

4. Se publica un EventoCambioEstadoSolicitud.
5. Se invoca el notificador externo.

Listing 8: Caso exitoso de cambio de estado usando Mockito

```
@Test
public void cambiarEstadoDebeGuardarPublicarEventoYNotificar() throws Exception {
    Estudiante estudiante = new Estudiante("Ana Torres", "ana@universidad.cl");
    Solicitud solicitud = new Solicitud(estudiante, "Certificado", "Solicitud valida");
    asignarId(solicitud, 10L);

    SolicitudPort solicitudPort = mock(SolicitudPort.class);
    PublicadorEventosSolicitud publicador = mock(PublicadorEventosSolicitud.class);
    NotificadorExterno notificador = mock(NotificadorExterno.class);

    when(solicitudPort.buscarPorId(10L)).thenReturn(Optional.of(solicitud));
    when(solicitudPort.guardar(any(Solicitud.class)))
        .thenAnswer(invocation -> invocation.getArgument(0));

    SolicitudService service = new SolicitudService(publicador, notificador, solicitudPort);

    Solicitud resultado = service.cambiarEstado(10L, EstadoSolicitud.EN_REVISION);

    assertEquals(EstadoSolicitud.EN_REVISION, resultado.getEstado());
    verify(solicitudPort).guardar(solicitud);
    verify(publicador).publicar(any(EventoCambioEstadoSolicitud.class));
    verify(notificador).enviarCorreo(eq("ana@universidad.cl"), contains("Cambio"), contains("
        EN_REVISION"));
}
```

Para inspeccionar el evento publicado se utiliza `ArgumentCaptor`. Esto permite verificar que el evento no solo fue enviado, sino que contiene los datos correctos.

Listing 9: Captura del evento publicado

```
ArgumentCaptor<EventoCambioEstadoSolicitud> eventoCaptor =
    ArgumentCaptor.forClass(EventoCambioEstadoSolicitud.class);
verify(publicador).publicar(eventoCaptor.capture());

EventoCambioEstadoSolicitud evento = eventoCaptor.getValue();
assertEquals(Long.valueOf(10L), evento.getSolicitudId());
assertEquals(EstadoSolicitud.PENDIENTE, evento.getEstadoAnterior());
assertEquals(EstadoSolicitud.EN_REVISION, evento.getEstadoNuevo());
assertNotNull(evento.getFechaCambio());
```

5.3 Casos fallidos en SolicitudServiceMockitoTest

El README pide verificar que no se llame al servicio externo cuando la operación falla. Se implementaron tres fallos relevantes:

- Solicitud inexistente.
- Solicitud ya finalizada.
- Nuevo estado nulo.

El patrón de verificación es el siguiente:

Listing 10: Patrón de no interacción ante error

```
assertThrows(IllegalArgumentException.class,
    () -> service.cambiarEstado(99L, EstadoSolicitud.EN_REVISION));

verify(solicitudPort).buscarPorId(99L);
verify(solicitudPort, never()).guardar(any(Solicitud.class));
verifyNoInteractions(publicador);
verifyNoInteractions(notificador);
```

Esto es fundamental: un test no solo debe probar lo que ocurre cuando todo sale bien, sino también que los efectos externos no se ejecutan si el caso de uso no se completa correctamente.

6 Tests arquitecturales con ArchUnit

Aunque ArchUnit no es un test unitario tradicional, el README lo solicita explícitamente como evidencia de recuperación arquitectural. Se implementaron reglas para las dependencias entre capas.

Listing 11: Reglas principales de arquitectura

```
@ArchTest
public static final ArchRule controllers_no_deben_depender_de_infraestructura =
    noClasses()
        .that().resideInAPackage("..controller..")
        .should().dependOnClassesThat()
        .resideInAnyPackage("..infra..");

@ArchTest
public static final ArchRule vista_no_debe_depender_de_infraestructura =
    noClasses()
        .that().resideInAPackage("..view..")
        .should().dependOnClassesThat()
        .resideInAnyPackage("..infra..");

@ArchTest
public static final ArchRule application_no_debe_depender_de_infraestructura =
    noClasses()
        .that().resideInAPackage("..application..")
        .should().dependOnClassesThat()
        .resideInAnyPackage("..infra..");
```

Estas reglas reflejan la inversión de dependencias: las capas superiores no deben conocer implementaciones concretas de infraestructura. La infraestructura puede depender de puertos o entidades, pero no al revés.

7 Tests de mensajería con Vert.x

7.1 VertxPublicadorEventosSolicitudTest

El README exige mantener o mejorar una parte orientada a mensajes. Para ello se prueba que VertxPublicadorEventosSolicitud publique un payload en el EventBus.

Listing 12: Verificación de publicación en EventBus

```

vertx.eventBus().consumer(VertxPublicadorEventosSolicitud.ADDRESS, message -> {
    payloadRecibido.set((JsonObject) message.body());
    latch.countDown();
});

VertxPublicadorEventosSolicitud publicador = new VertxPublicadorEventosSolicitud(vertx);
publicador.publicar(new EventoCambioEstadoSolicitud(
    10L,
    EstadoSolicitud.PENDIENTE,
    EstadoSolicitud.EN_REVISION,
    fecha
));

assertTrue(latch.await(3, TimeUnit.SECONDS));
assertEquals("PENDIENTE", payload.getString("estadoAnterior"));
assertEquals("EN_REVISION", payload.getString("estadoNuevo"));

```

Se usa CountdownLatch porque la entrega del mensaje es asíncrona.

7.2 HistorialEventosVerticleTest

También se verifica que un receptor deje evidencia de recepción. Para evitar depender de SQLite/JPA en esta prueba, se creó un repositorio en memoria extendiendo HistorialEventoRepository y sobrescribiendo guardar(...).

Listing 13: Repositorio en memoria para probar el receptor

```

private static class RepositorioHistorialEnMemoria extends HistorialEventoRepository {
    private final List<HistorialEvento> guardados = new ArrayList<>();
    private final CountdownLatch latch = new CountdownLatch(1);

    @Override
    public HistorialEvento guardar(HistorialEvento evento) {
        guardados.add(evento);
        latch.countDown();
        return evento;
    }
}

```

Esta decisión mantiene la prueba enfocada en la recepción del evento, no en detalles internos de Hibernate.

8 Estructura final de archivos de test

Listing 14: Estructura propuesta de tests

```

src/test/java/cl/ucn/solicitudes/domain/model/EstudianteTest.java
src/test/java/cl/ucn/solicitudes/domain/model/SolicitudDomainTest.java
src/test/java/cl/ucn/solicitudes/domain/model/EstadoSolicitudTest.java
src/test/java/cl/ucn/solicitudes/application/EstudianteServiceTest.java
src/test/java/cl/ucn/solicitudes/application/EventoCambioEstadoSolicitudTest.java
src/test/java/cl/ucn/solicitudes/application/SolicitudServiceMockitoTest.java
src/test/java/cl/ucn/solicitudes/architecture/ArquitecturaMvcTest.java
src/test/java/cl/ucn/solicitudes/infra/messaging/VertxPublicadorEventosSolicitudTest.java
src/test/java/cl/ucn/solicitudes/infra/messaging/HistorialEventosVerticleTest.java

```


9 Comando de ejecución y cobertura

El proyecto está preparado para ejecutarse con Maven:

Listing 15: Ejecución de tests y JaCoCo

```
mvn clean verify
```

El reporte de cobertura de JaCoCo se genera normalmente en:

Listing 16: Ubicación esperada del reporte JaCoCo

```
target/site/jacoco/index.html
```

La configuración del `pom.xml` exige al menos 80 % de cobertura de líneas a nivel de bundle durante la fase `verify`. Por tanto, si alguna clase no queda cubierta, Maven debe fallar en la meta `jacoco:check`.

Nota de verificación

En el entorno de generación de este informe no estaba disponible el comando `mvn`; por ello, la suite fue preparada y revisada estáticamente, pero la ejecución efectiva debe realizarse en IntelliJ IDEA o en una terminal con Maven instalado mediante `mvn clean verify`.

10 Conclusión

La suite implementada cubre los casos mínimos sugeridos por el README y agrega verificaciones adicionales útiles: nulos, espacios en blanco, estados finales diferenciados, delegación de puertos, ausencia de interacciones externas ante fallos, reglas arquitecturales y mensajería asíncrona con Vert.x.

El punto metodológico más importante es que los tests no duplican innecesariamente las reglas: el dominio valida reglas internas, la capa de aplicación valida coordinación del caso de uso y Mockito verifica interacción con dependencias. Con esta distribución, la suite queda alineada con una arquitectura más mantenible y con la intención pedagógica de la evaluación.